PALANTIR

An open-source digital twin bridging astrodynamics and mission control.

APRIL 2026

Where this comes from

10+ years of mission-critical Java systems engineering: Telco SDN · FinTech transactional systems

Currently shipping: a distributed booking platform with protocol translation and a spatial data normalisation engine *(Coderama, 2022 – current)*

MSc thesis (UMB, 2017): C++/OpenMP implementation of a peer-reviewed heavy-ion event-classification method — translating published science into working code

---

I stopped reading about space and started building.

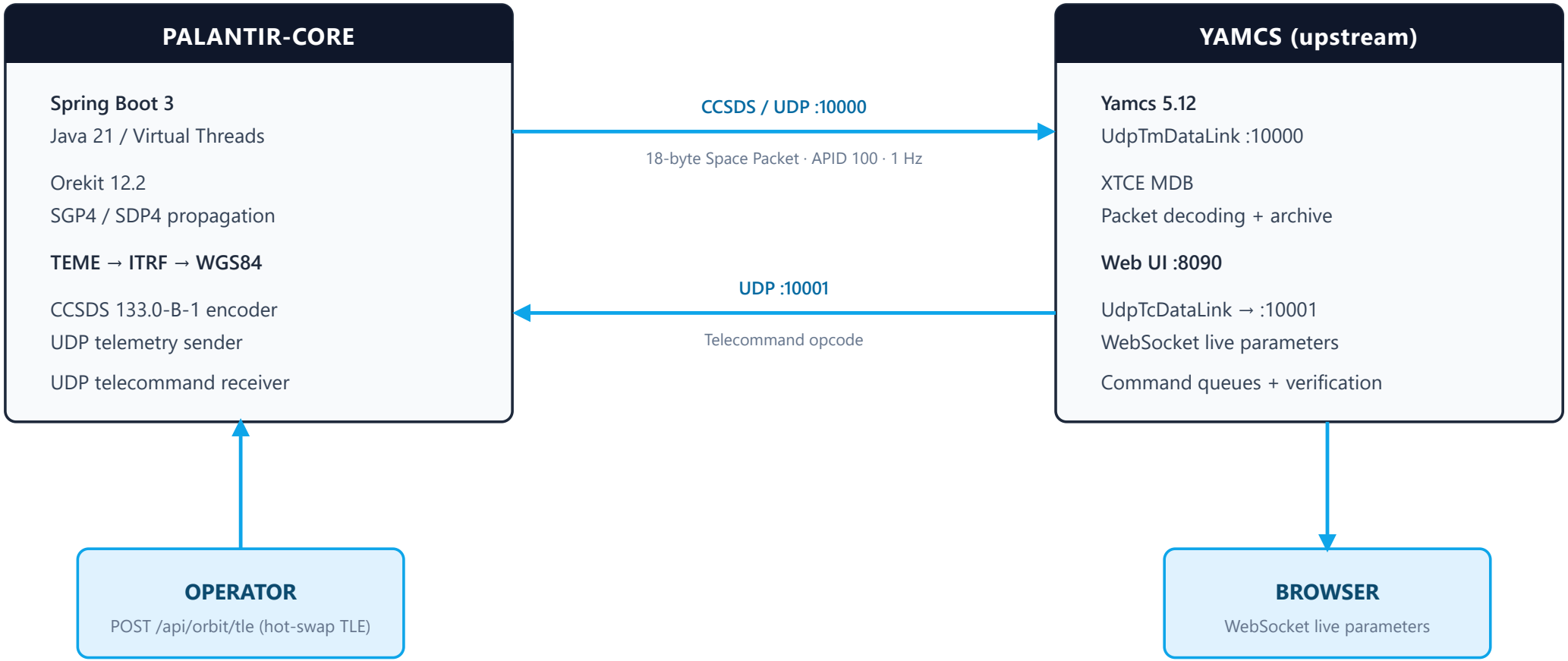
Why the ground segment

TELCO SDN *(where I ship)*	SPACE GROUND SEGMENT
YANG (RFC 7950)	**XTCE** (CCSDS 660.0-B-2)
NETCONF / RESTCONF (RFC 6241 / 8040)	**CCSDS Space Packet** (CCSDS 133.0-B-1)
OpenDaylight (Apache Karaf)	**Yamcs** (OSGi-style services)
Shipped to 99.999% Telco SLAs	Mission-critical TM/TC loops

Seven of those years in the left column made the right column look familiar.

What it is

Three components, one Docker Compose command. Fully open-source stack.



Where it is today

Not a slide deck. A working system you can run today.

```
$ docker compose up --build
```

Within a minute of a warm start:

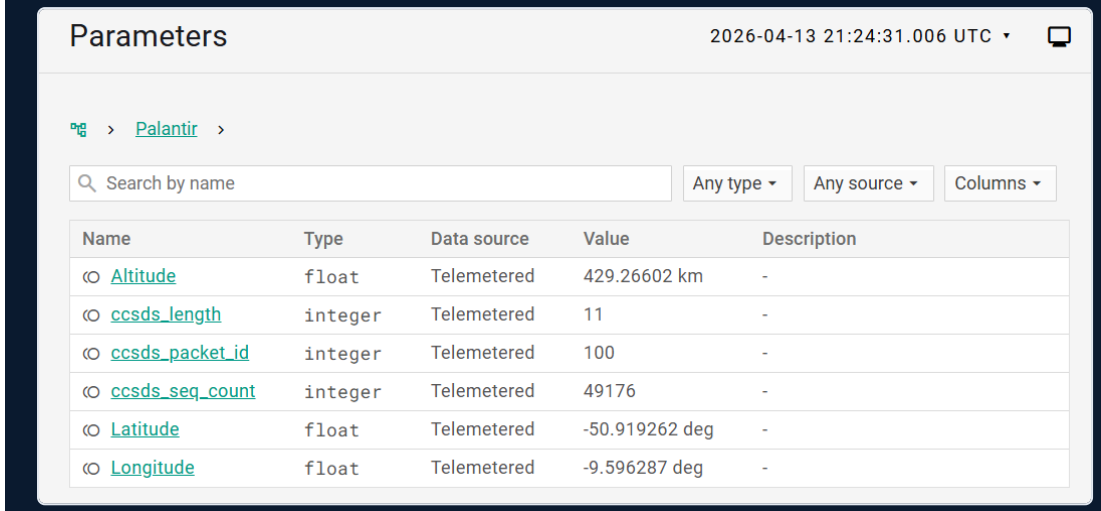
- Default ISS TLE loads automatically
- Yamcs Web UI on `http://localhost:8090`
- Three parameters streaming at 1 Hz: `/Palantir/Latitude` · `/Palantir/Longitude` · `/Palantir/Altitude`
- Persistent archive on Docker volume — survives restart

Engineering quality:

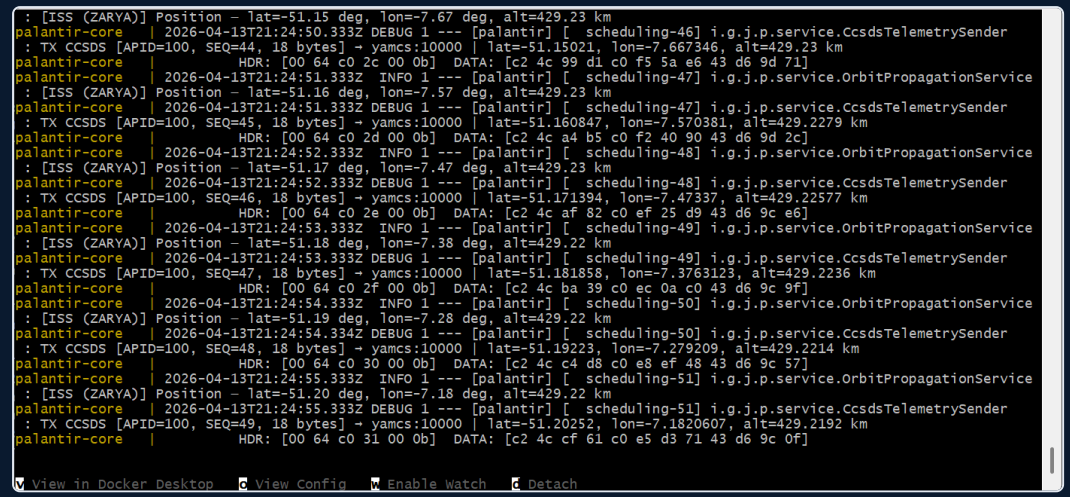
- JUnit 5 / Mockito / AssertJ — tests verifying TLE ingestion, orbital propagation plausibility, and Spring context integrity
- JaCoCo coverage report
- Javadoc on every main class

Active development since February 2026 · MIT license

What you'll see



Live parameters at 21:24:31 UTC



Raw CCSDS Space Packets, 1 Hz cadence

Standards-compliant CCSDS Space Packets — the same binary protocol used by ESA missions — are decoded in real time by Yamcs at 1 Hz cadence. Three orbital parameters (latitude, longitude, altitude) are streamed live from SGP4 propagation.

One header decoded: `00 64` = APID 100 · `c0 2c` = grouping flags + 14-bit counter 44 · `00 0b` = data length 11 (payload is 12 B).

* `ccsds_seq_count` *in the UI shows the raw 16-bit field (grouping flags + 14-bit counter), so UI values > 16383 are expected.*

What I want to build

The Demo Day target: one end-to-end Automated Collision Avoidance flow, live on stage from a clean `$ docker compose up --build`. Sequential build: screening first, commanding second, physics reaction last — each layer testable independently.

1. CONJUNCTION SCREENING

- Standalone Orekit batch job · CelesTrak GP catalog (OMM XML)
- 7-day propagation window · Monte Carlo Probability of Collision
- Top events posted to the Yamcs event log

2. OPERATOR-APPROVED COMMANDING

- `FireThruster` XTCE telecommand
- Critical-significance command queue
- No autonomous burn ever flies

3. VISIBLE PHYSICS REACTION

- Operator clicks APPROVE → impulsive Δv applied to the orbit
- Ground track visibly shifts within one orbital period
- *A closed loop, made visible.*

If time allows beyond the September critical path: synthetic telemetry generation + LSTM autoencoder anomaly detection with ONNX export.

What I do not know yet

I built Palantir to test whether 10+ years of mission-critical systems engineering can translate into the space sector. **That is precisely why I am applying.**

Two paths I would be happy with, either or both:

- **Path A — own project trajectory.** If Palantir proves viable during the programme, continue it as an independent initiative beyond September. If the traction is real, explore ESA BIC Slovakia or similar incubation as a longer-term step.
- **Path B — sector transition.** Use the six months as a structured entry into the European space industry. Build the network, understand how operators and integrators hire, and position myself for a role in mission-critical space software.

Open questions either path would help me answer:

- What are the real bottlenecks in European ground segment operations today? Where does the current toolchain break or force manual workarounds?
- Where does the open-source stack (Yamcs, Orekit, OpenC3) stop short of production needs — and where is institutional modernisation blocked by budget, qualification, or risk aversion?

What I want to learn from the programme:

Where the open-source ground segment approach has real commercial traction — and where it does not. That answer shapes which of the two paths above I take.

My commitments in return:

- **Working prototype available from day one.** Live demo at every Advisory Board checkpoint, on the reviewer's laptop.
- **All artefacts open-sourced under MIT** — no IP friction, no surprises.
- **Genuinely open to mentor guidance.** The Automated Collision Avoidance flow is my current plan for Demo Day, but I will adapt the approach if Advisory Board feedback points to something stronger.